

Architectures (I)



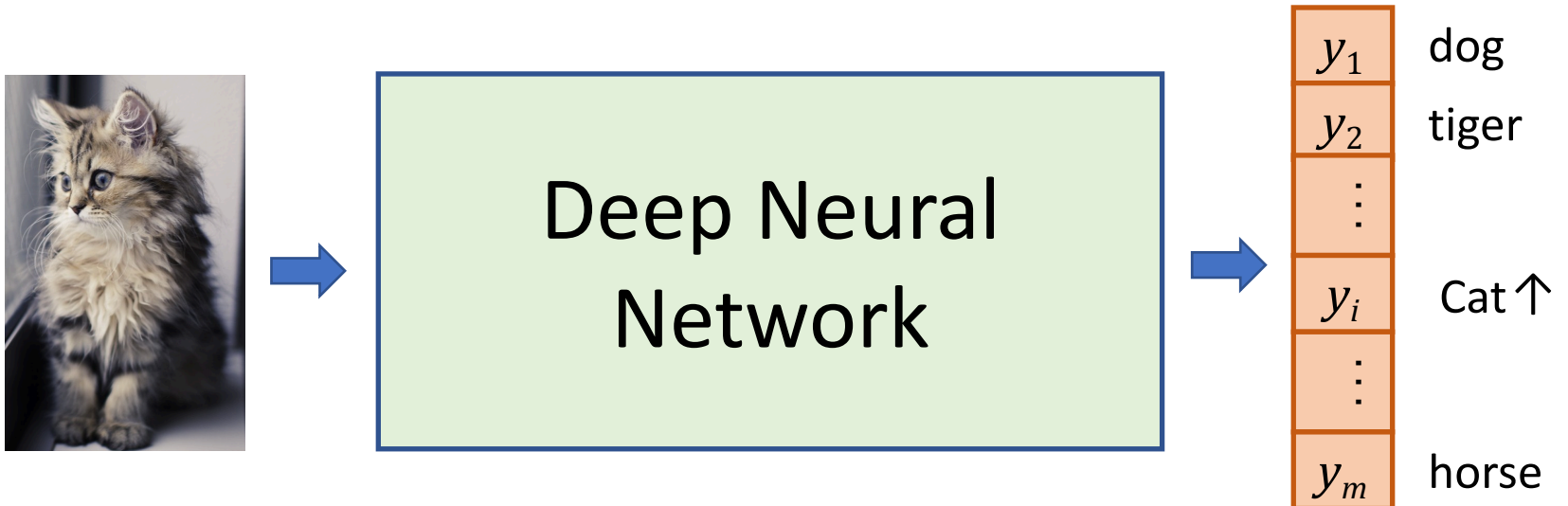
Guosheng Hu

Source from Farnoosh Heidarivincheh

Convolutional Neural Networks

So far, we have discussed

- Neural networks with Fully-Connected layers
- The optimization process / How the network is trained



Outline

- Convolutional Neural Networks
 - Different Layers
 - Popular Architectures
- Other Popular Neural Networks
 - Recurrent Neural Networks
 - Long Short-Term Memory Networks
 - Auto-Encoders
 - Generative Adversarial Networks

Convolutional Neural Networks

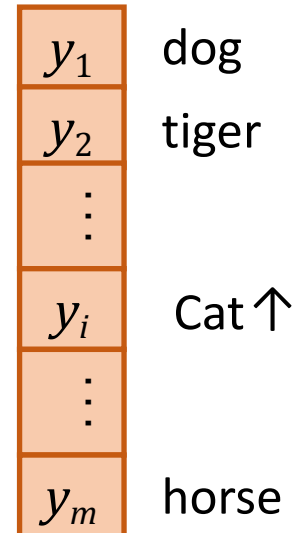
Convolutional Neural Networks

Image classification with **CNN**

- No need to pre-process images
- CNNs outperform traditional approaches



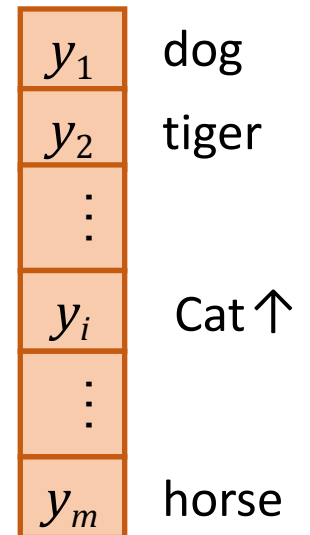
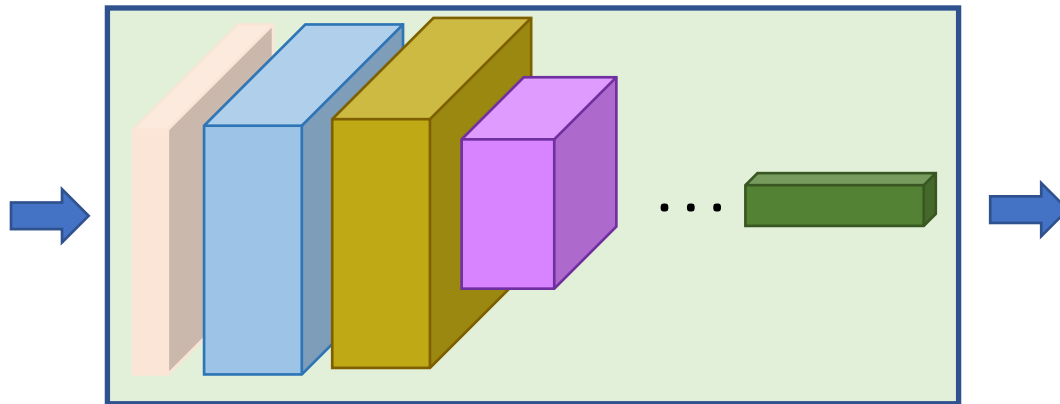
Convolutional
Neural Network
(CNN)



Convolutional Neural Networks

Image classification with **CNN** - Layers in a CNN:

- Input
- Convolutional
- Activation (ReLU, ...)
- Pooling
- Fully-Connected (FC)
- Drop-out
- Batch Normalisation (BN), ...



Convolution Operation

Source layer

5	2	6	8	2	0	1	2
4	3	4	5	1	9	6	3
3	9	2	4	7	7	6	9
1	3	4	6	8	2	2	1
8	4	6	2	3	1	8	8
5	8	9	0	1	0	2	3
9	2	6	6	3	6	2	1
9	8	8	2	6	3	4	5

Convolutional kernel

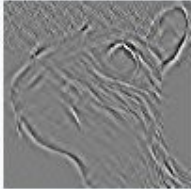
-1	0	1
2	1	2
1	-2	0

Destination layer

	5						

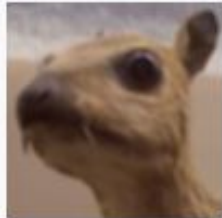
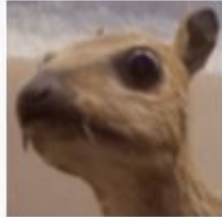
$$\begin{aligned} &(-1 \times 5) + (0 \times 2) + (1 \times 6) + \\ &(2 \times 4) + (1 \times 3) + (2 \times 4) + \\ &(1 \times 3) + (-2 \times 9) + (0 \times 2) = 5 \end{aligned}$$

Convolutional Kernels

Operation	Kernel ω	Image result $g(x,y)$
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or edge detection	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	

[https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing))

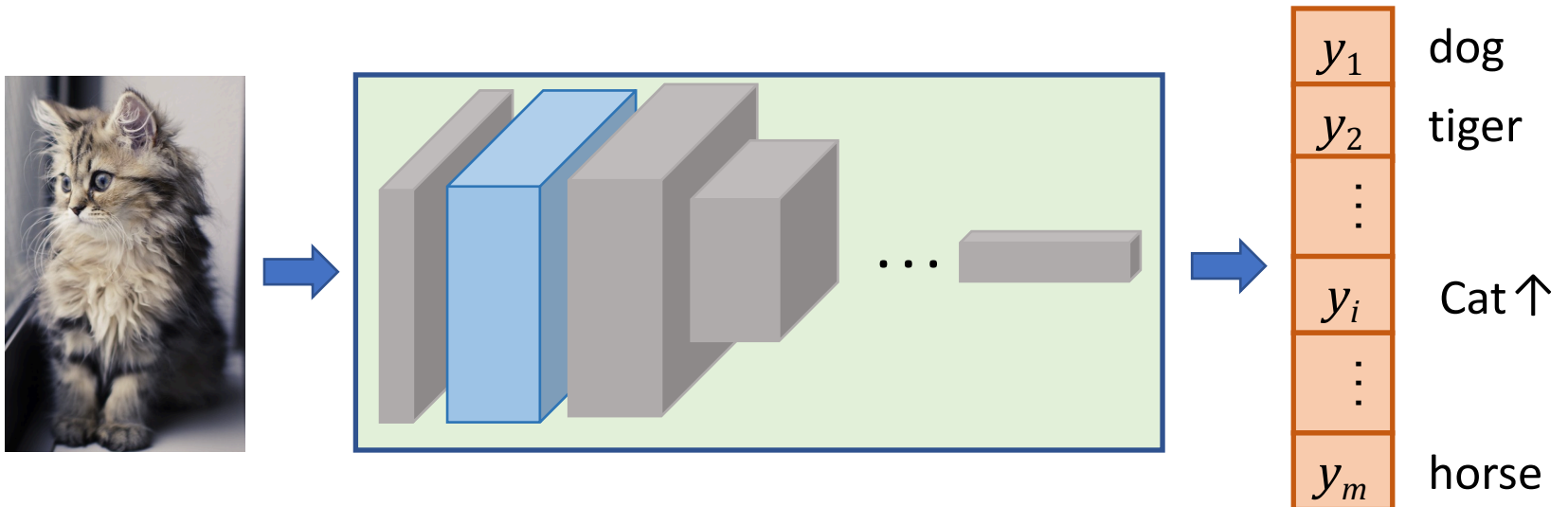
Convolutional Kernels

Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur 3 x 3 (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

Convolutional Neural Networks

Convolutional layer

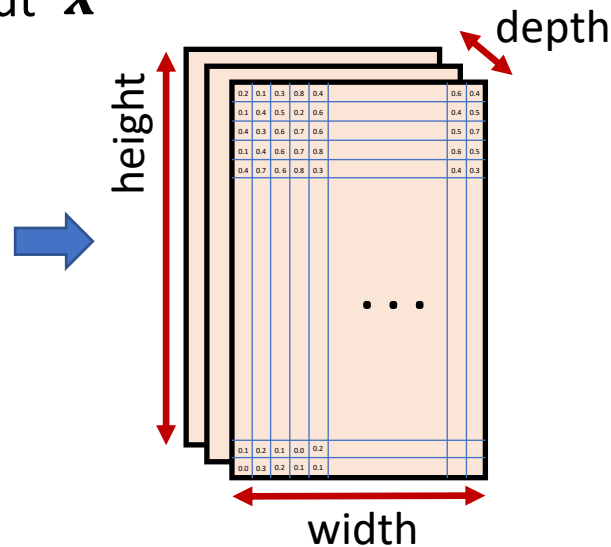
- It is the main building block of a CNN
- its neurons can have more than one dimension
- It captures the correlation among neighboring data points



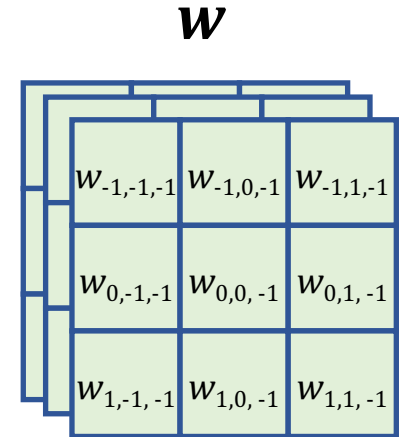
Convolutional Neural Networks

Convolutional layer

RGB image input $\mathbf{X}$



Convolution kernel $\mathbf{W}$

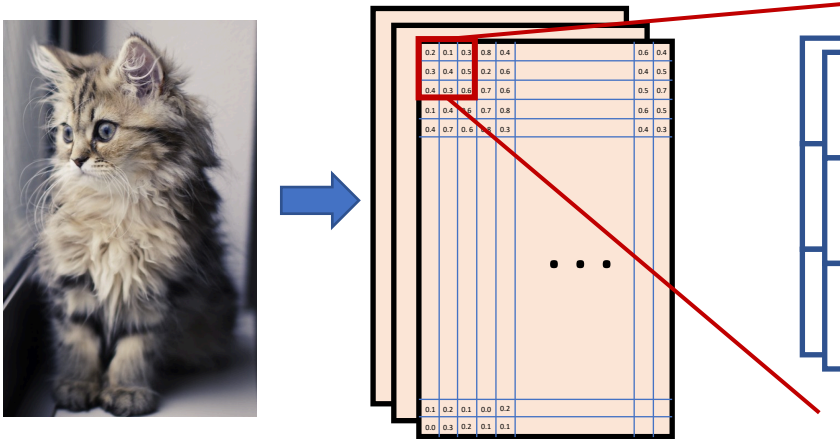


Kernel size: $3 \times 3 \times 3$

Convolutional Neural Networks

Convolutional layer

RGB image input $\mathbf{X}$



0.2	0.1	0.3
0.3	0.4	0.5
0.4	0.3	0.6

Convolution kernel

$\mathbf{W}$

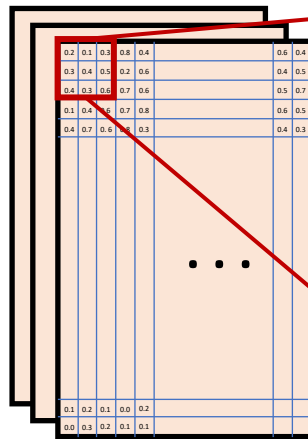
$w_{-1,-1,-1}$	$w_{-1,0,-1}$	$w_{-1,1,-1}$
$w_{0,-1,-1}$	$w_{0,0,-1}$	$w_{0,1,-1}$
$w_{1,-1,-1}$	$w_{1,0,-1}$	$w_{1,1,-1}$

Kernel size: 3×3×3

Convolutional Neural Networks

Convolutional layer

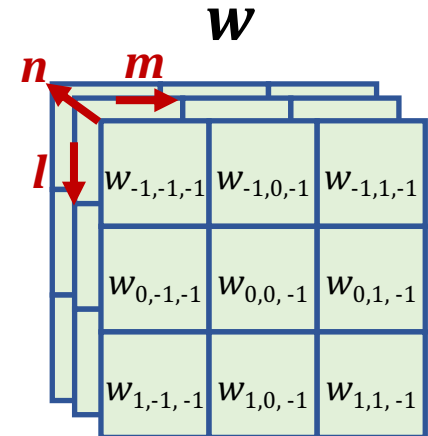
RGB image input $\mathbf{X}$



0.2	0.1	0.3
0.3	0.4	0.5
0.4	0.3	0.6

building block of NN,
a simplified (local) FC layer

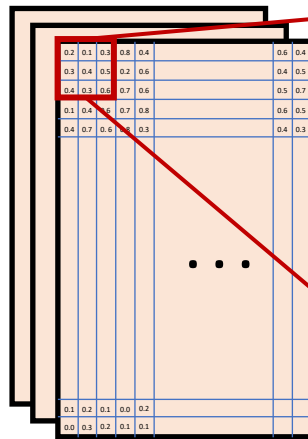
$$w_b + \sum_{l,m,n=-1}^1 x_{i+l,j+m,k+n} w_{l,m,n}$$



Convolutional Neural Networks

Convolutional layer

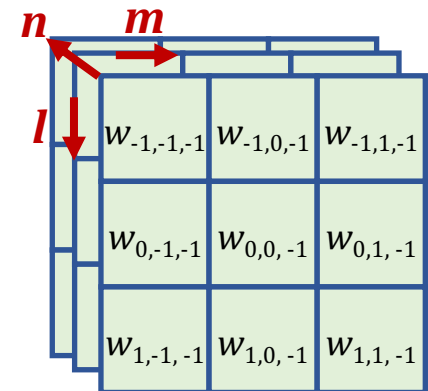
RGB image input $\mathbf{X}$



0.2	0.1	0.3
0.3	0.4	0.5
0.4	0.3	0.6

$$w_b + \sum_{l,m,n=-1}^1 x_{i+l,j+m,k+n} \underline{w_{l,m,n}}$$

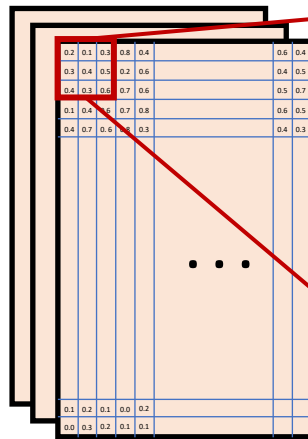
$\mathbf{w}$



Convolutional Neural Networks

Convolutional layer

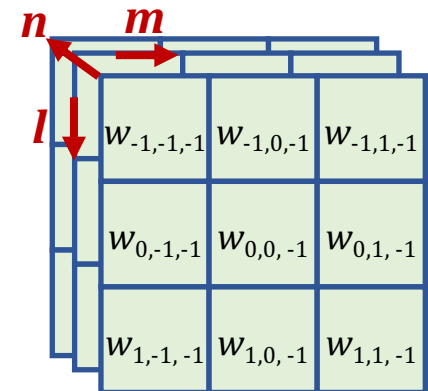
RGB image input $\mathbf{X}$



0.2	0.1	0.3
0.3	0.4	0.5
0.4	0.3	0.6

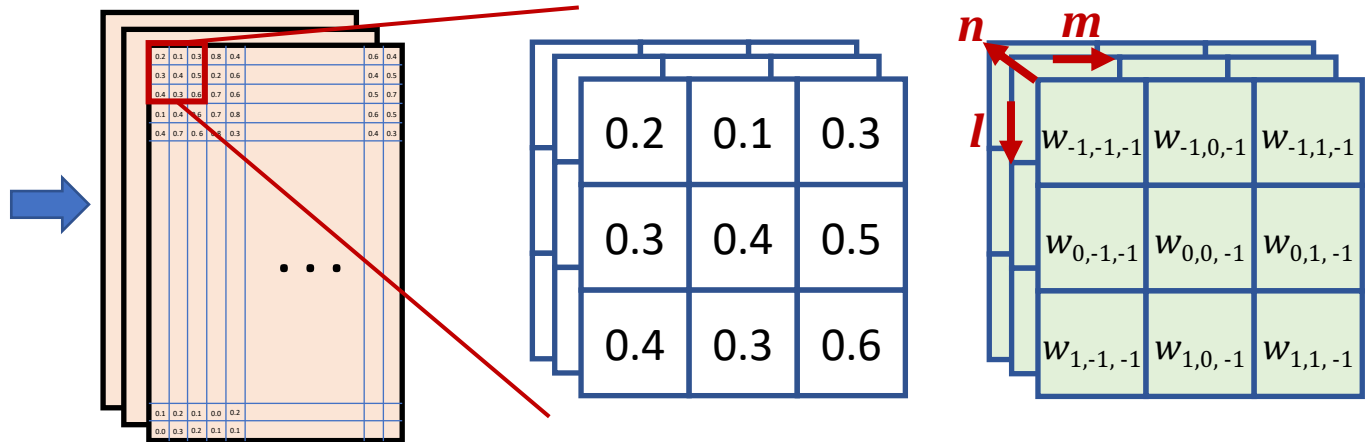
$$w_b + \sum_{l,m,n=-1}^1 \underline{x_{i+l,j+m,k+n}} w_{l,m,n}$$

$\mathbf{W}$



Convolutional Neural Networks

Convolutional layer

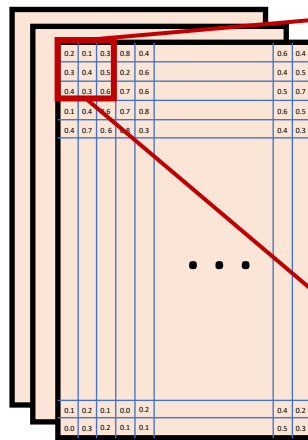
RGB image input $\mathbf{x}$ 

$$\text{bias } w_b + \sum_{l,m,n=-1}^1 x_{i+l,j+m,k+n} w_{l,m,n}$$

Convolutional Neural Networks

Convolutional layer

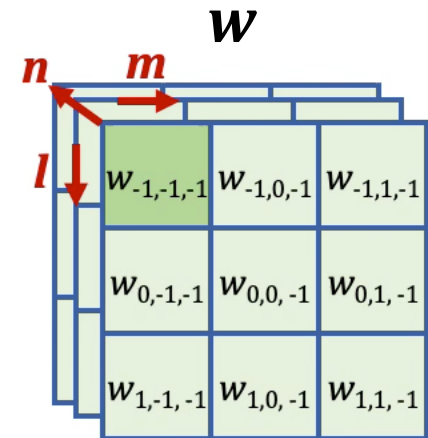
RGB image input $\mathbf{X}$



0.2	0.1	0.3
0.1	0.4	0.5
0.4	0.3	0.6

$$w_b + 0.2 \times w_{-1,-1,-1}$$

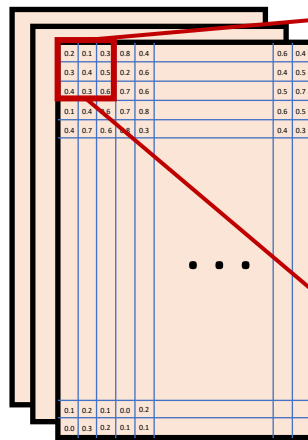
$$w_b + \sum_{l,m,n=-1}^1 x_{i+l,j+m,k+n} w_{l,m,n}$$



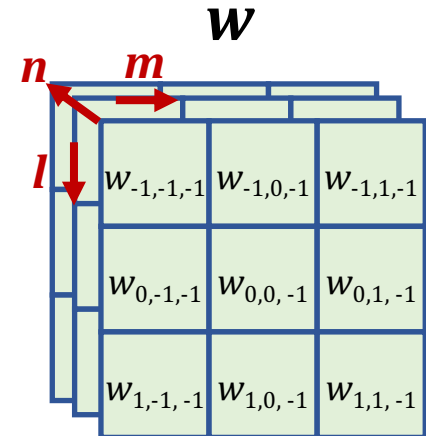
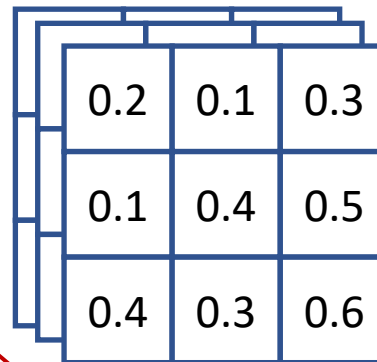
Convolutional Neural Networks

Convolutional layer

RGB image input $\mathbf{X}$



$$w_b + \sum_{l,m,n=-1}^1 x_{i+l,j+m,k+n} w_{l,m,n}$$



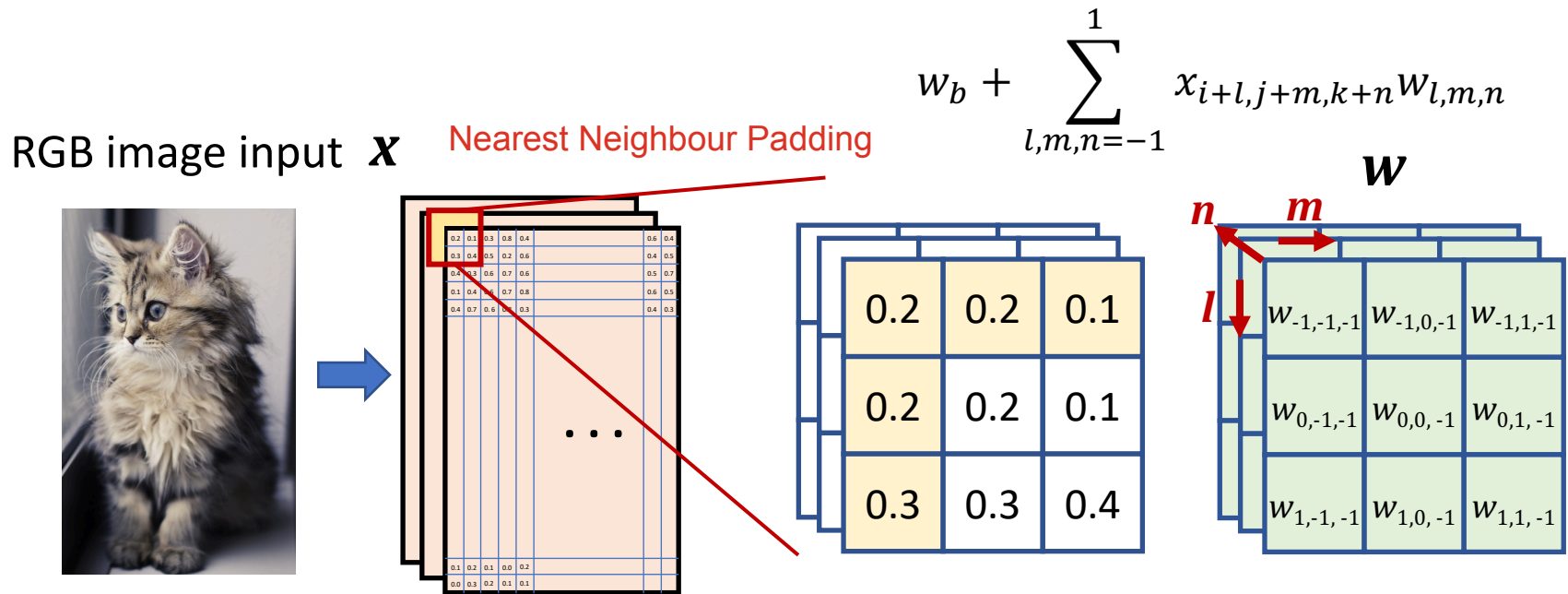
This is repeated for every pixel of the image (if *stride*=1)

Image is padded for the bordering pixels

So that the output is the same size as its input

Convolutional Neural Networks

Convolutional layer



This is repeated for every pixel of the image (if *stride*=1)

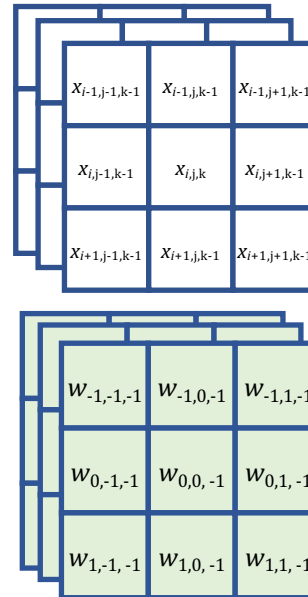
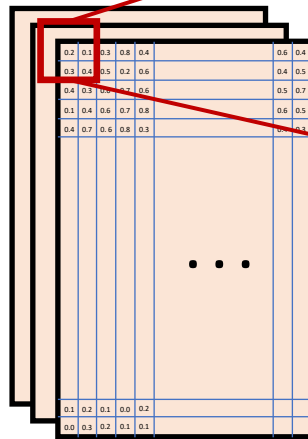
Image is padded for the bordering pixels

So that the output is the same size as its input

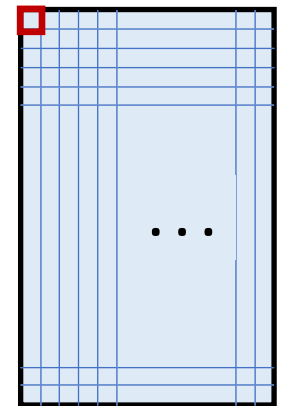
Convolutional Neural Networks

Convolutional layer

RGB image input



Conv layer output



This is repeated for every pixel of the image (if *stride*=1)

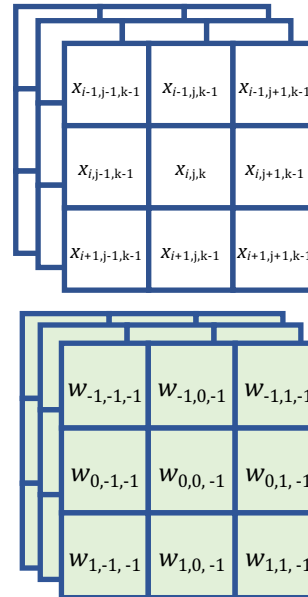
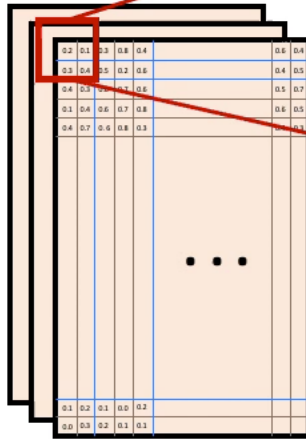
Image is padded for the bordering pixels

So that the output is the same size as its input

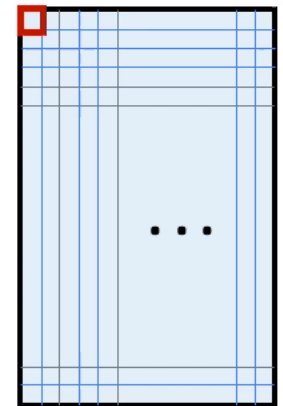
Convolutional Neural Networks

Convolutional layer

RGB image input



Conv layer output



This is repeated for every pixel of the image (if *stride*=1)

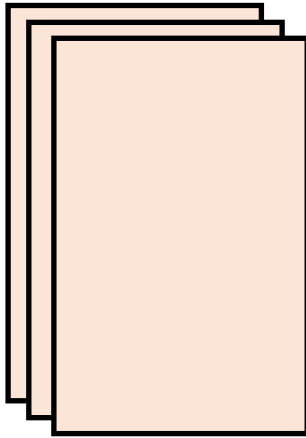
Image is padded for the bordering pixels

So that the output is the same size as its input

Convolutional Neural Networks

Convolutional layer

RGB image input $\mathbf{x}$



$\mathbf{w}^{(1)}$



$\mathbf{w}^{(2)}$



$\mathbf{w}^{(3)}$

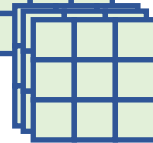


$\vdots$

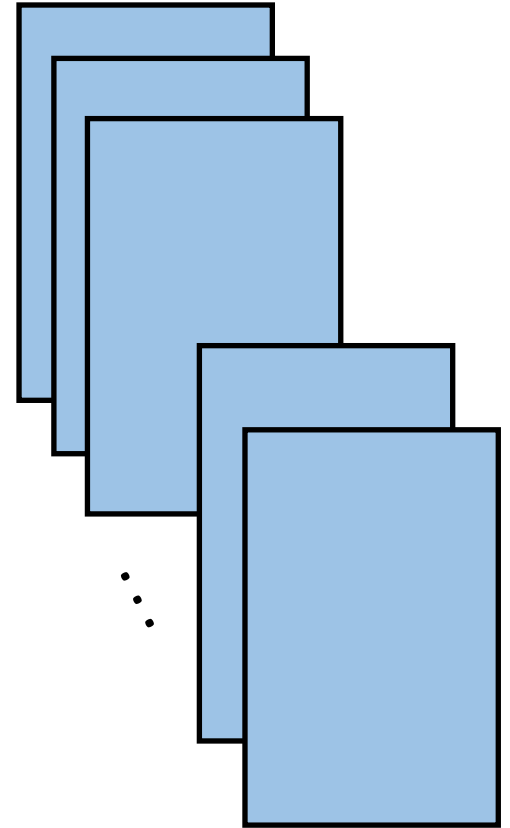
$\mathbf{w}^{(c-1)}$



$\mathbf{w}^{(c)}$



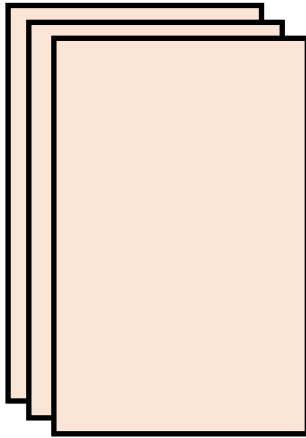
Conv layer output



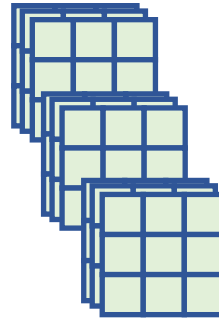
Convolutional Neural Networks

Convolutional layer

RGB image input $\mathbf{x}$



$\mathbf{w}^{(1)}$



$\mathbf{w}^{(2)}$

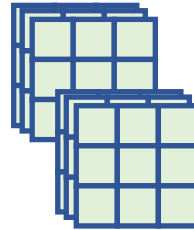


$\mathbf{w}^{(3)}$



...

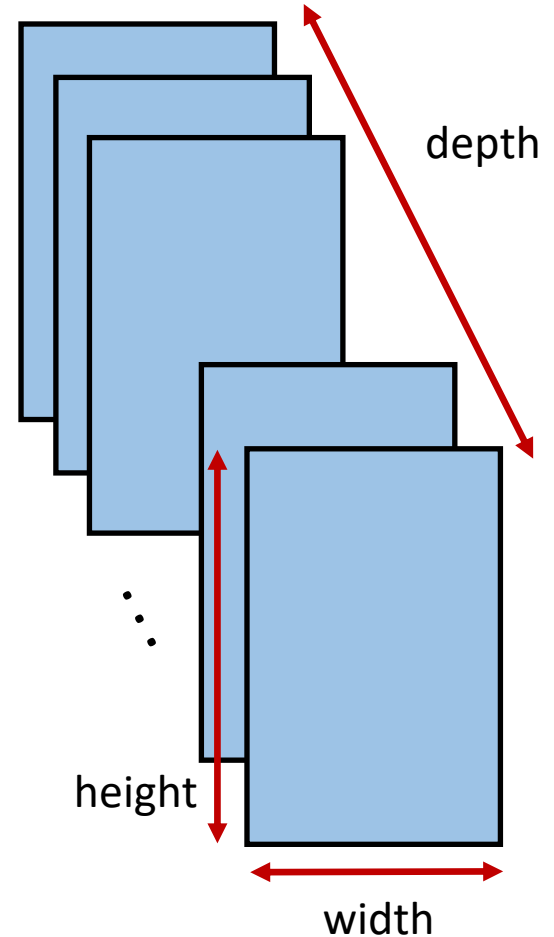
$\mathbf{w}^{(c-1)}$



$\mathbf{w}^{(c)}$



Conv layer output



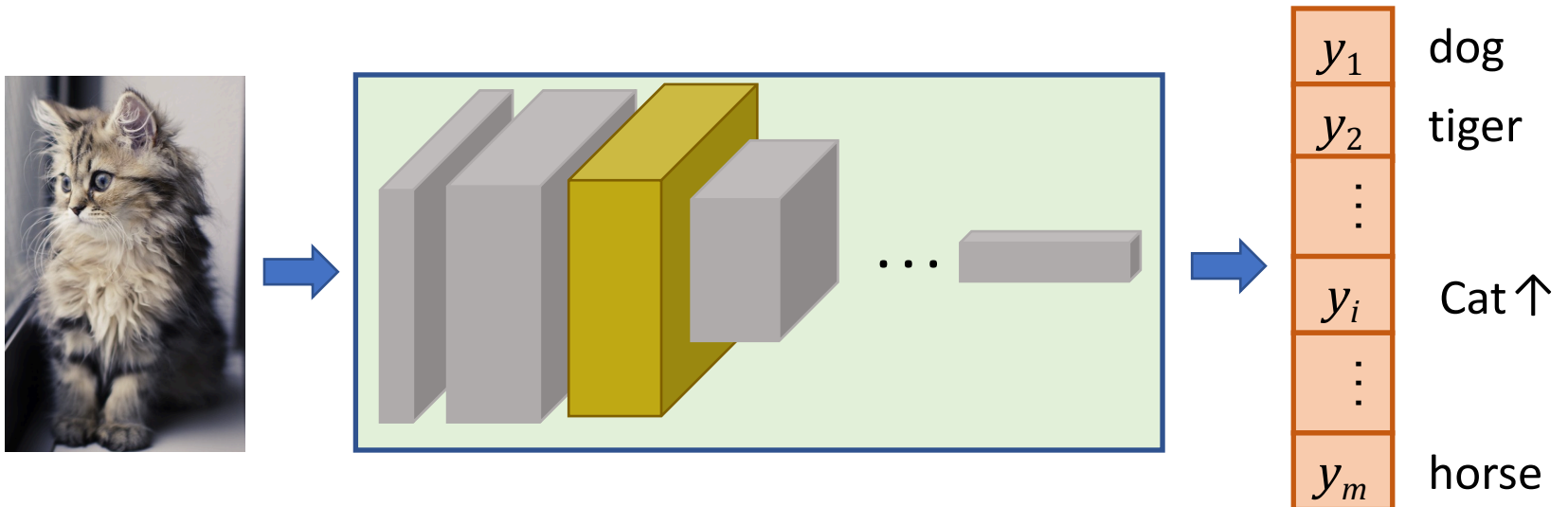
This can be used as the
input to another layer



Convolutional Neural Networks

Activation layer

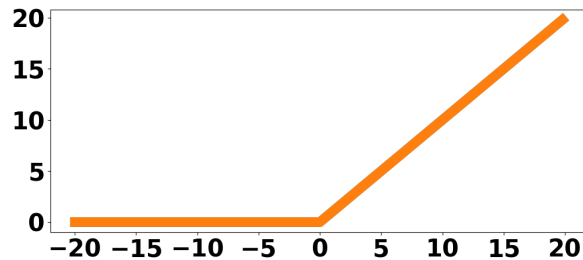
- It introduces non-linearity to the network
- Non-linearity helps the network learn complex patterns
- The most common activations are Sigmoid, ReLU and Tanh



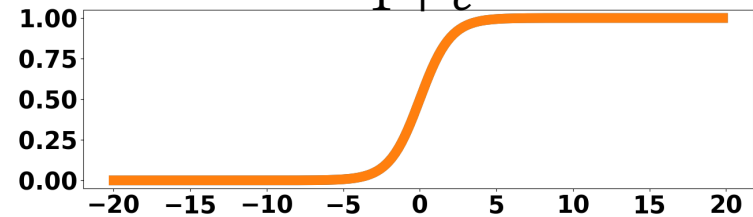
Convolutional Neural Networks

Activation layer

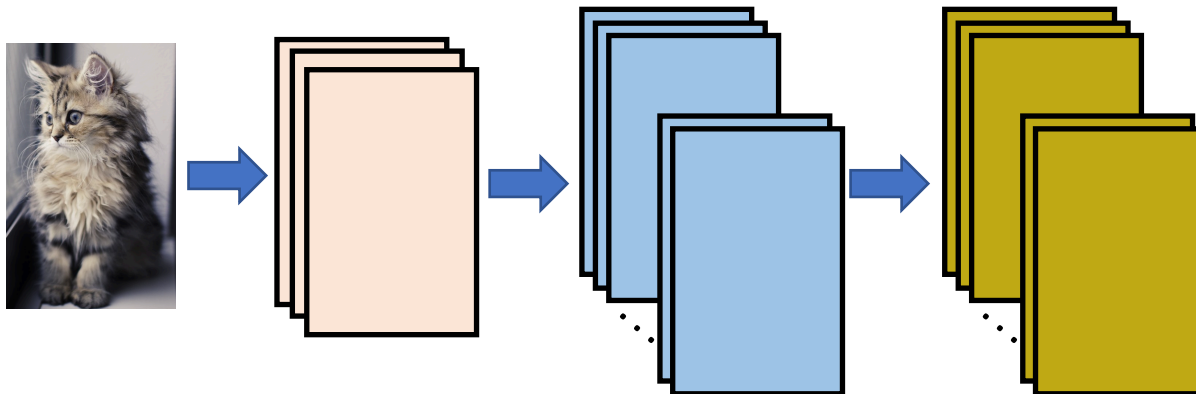
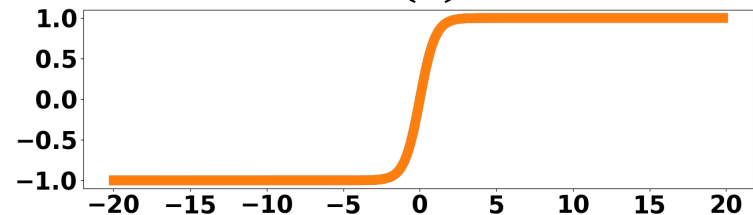
$ReLU(x) = \max(0, x)$
(Rectified Linear Unit)



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



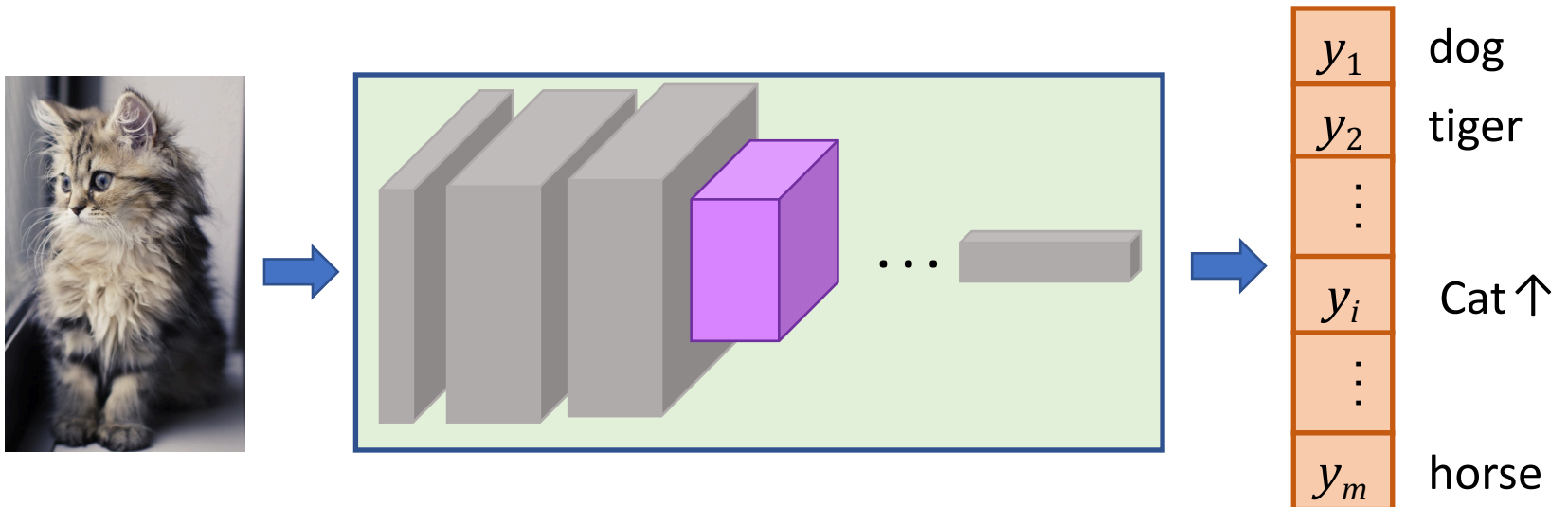
$$\tanh(x)$$



Convolutional Neural Networks

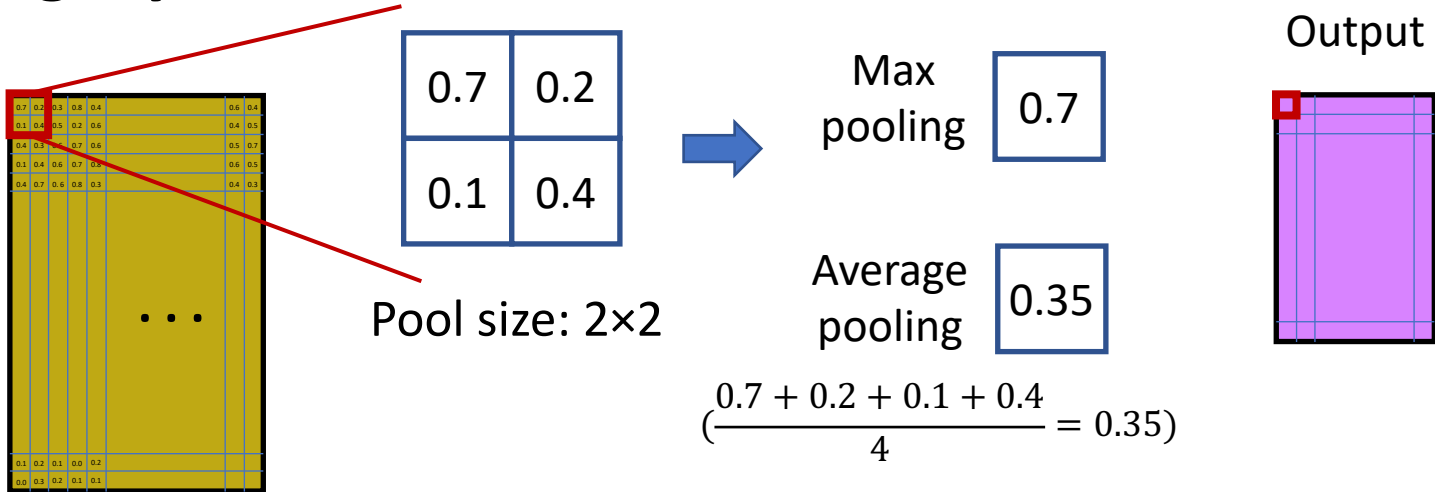
Pooling layer

- It down-samples its input
- It reduces the network sensitivity to the features' location
- Two common pooling layers are average and max pooling



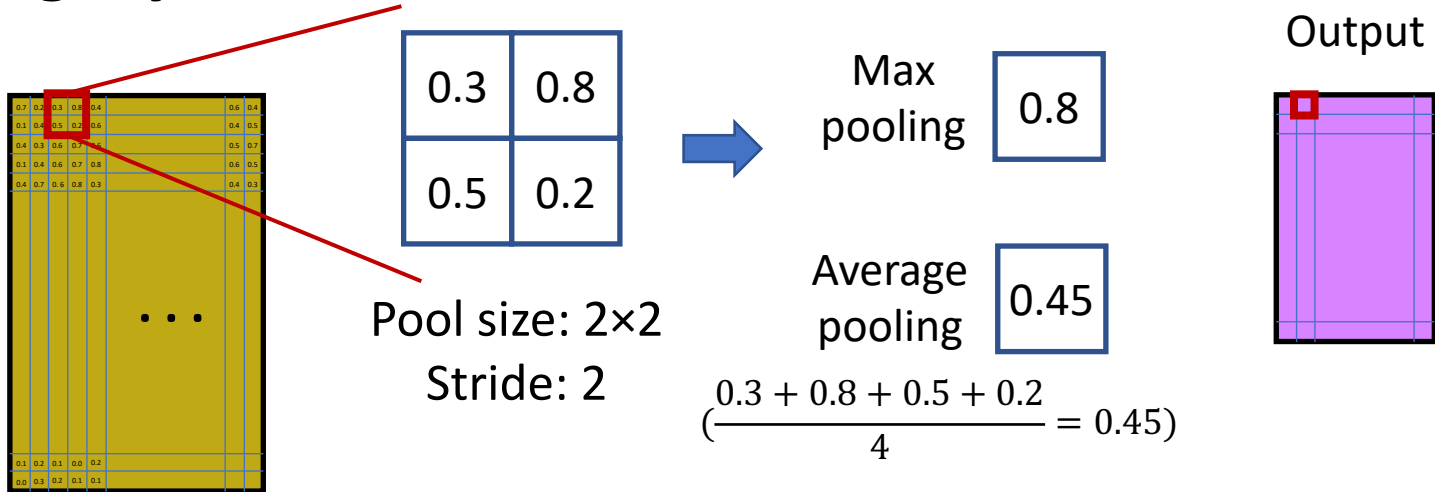
Convolutional Neural Networks

Pooling layer



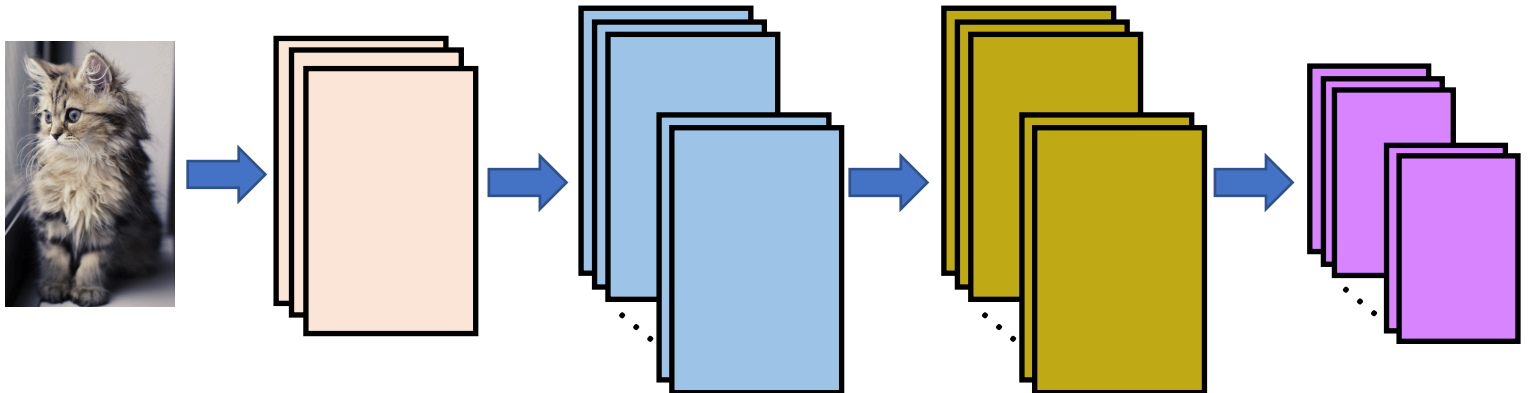
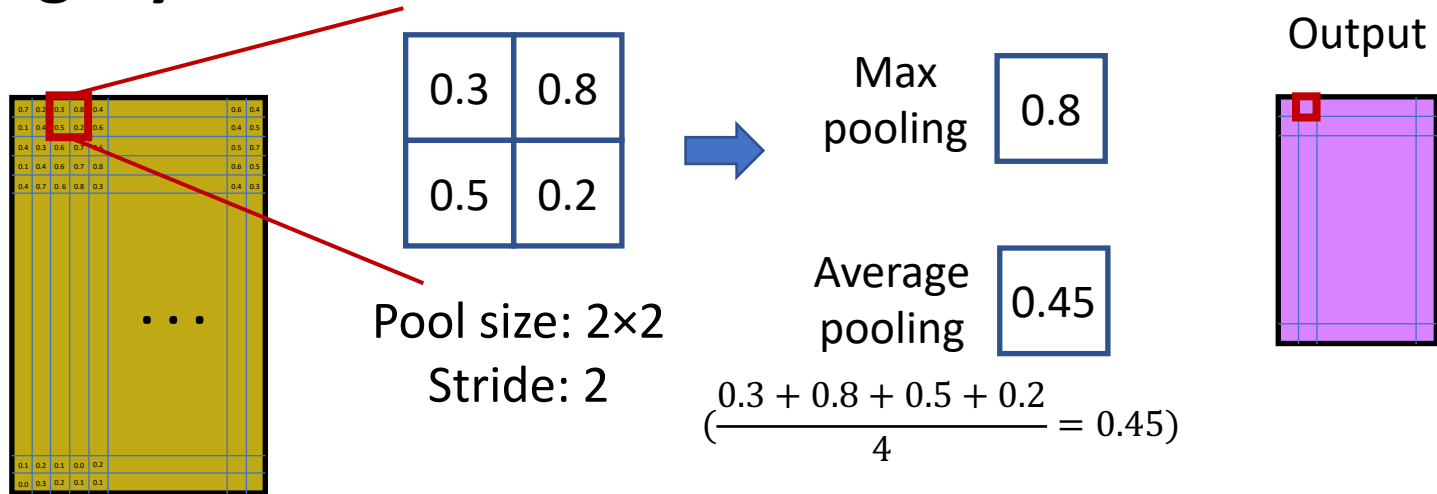
Convolutional Neural Networks

Pooling layer



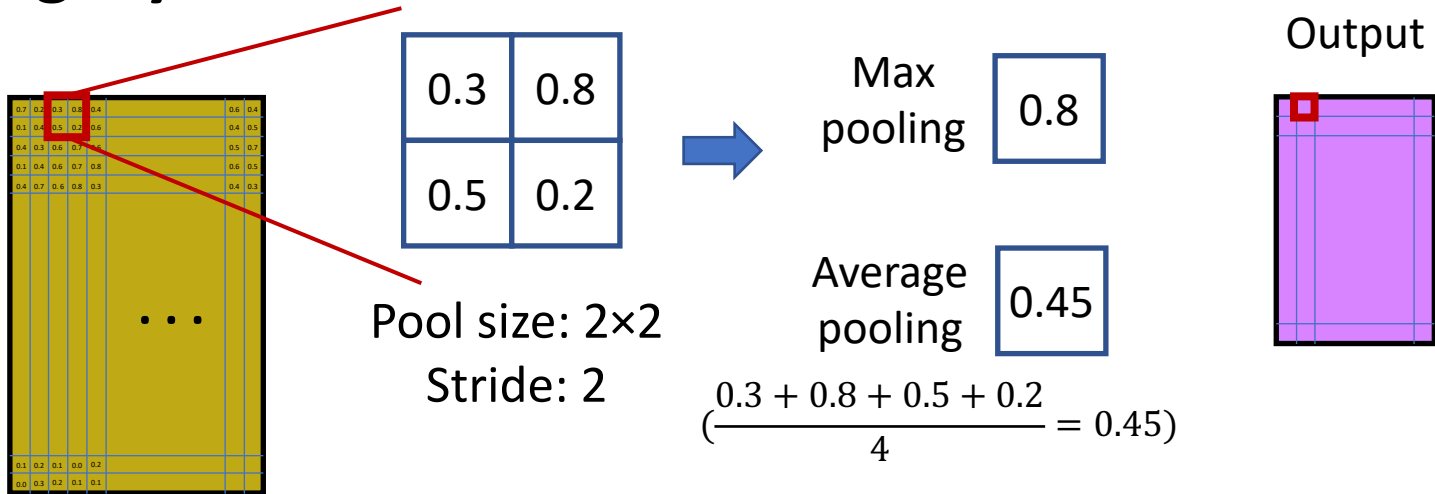
Convolutional Neural Networks

Pooling layer



Convolutional Neural Networks

Pooling layer

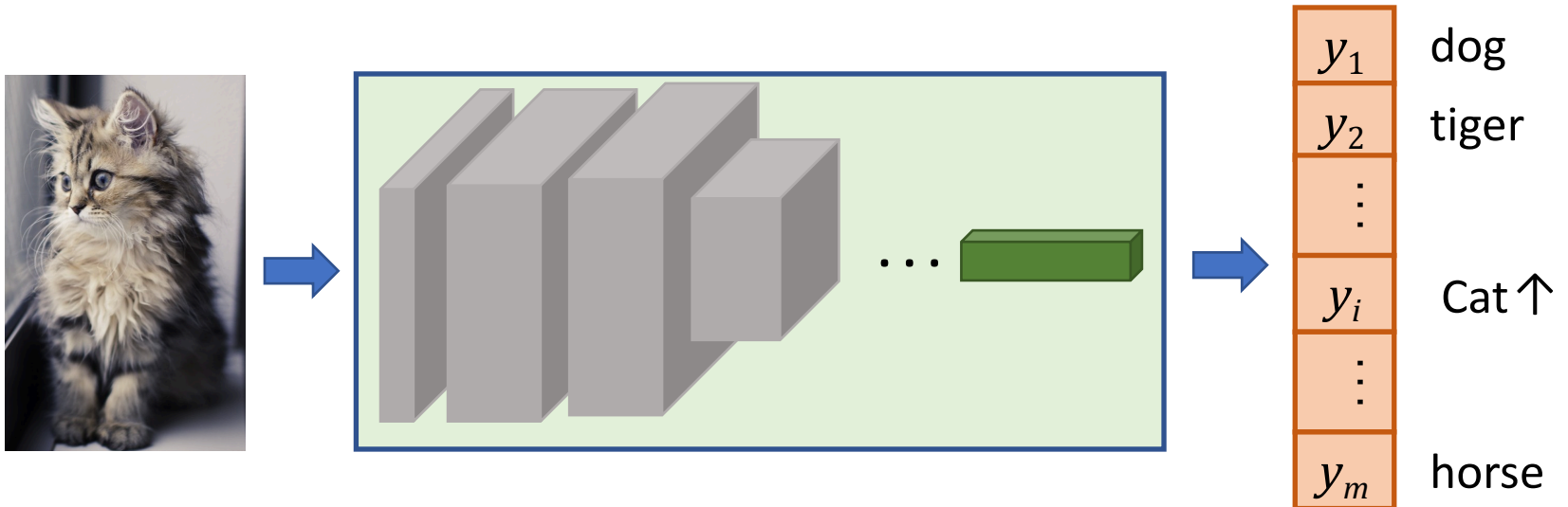


Pooling layer is not learnable

Convolutional Neural Networks

Fully-Connected layer

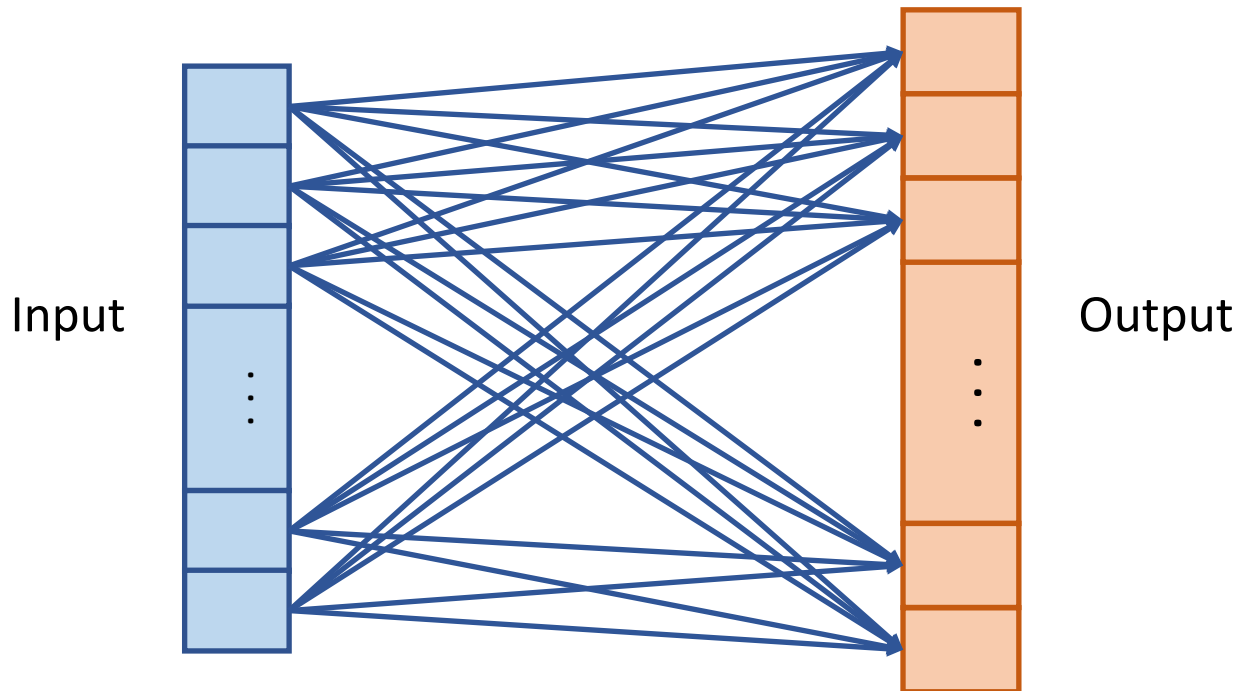
- It connects all its inputs to all the outputs
- It is usually used as the last layers of a network (before classification)



Convolutional Neural Networks

Fully-Connected layer

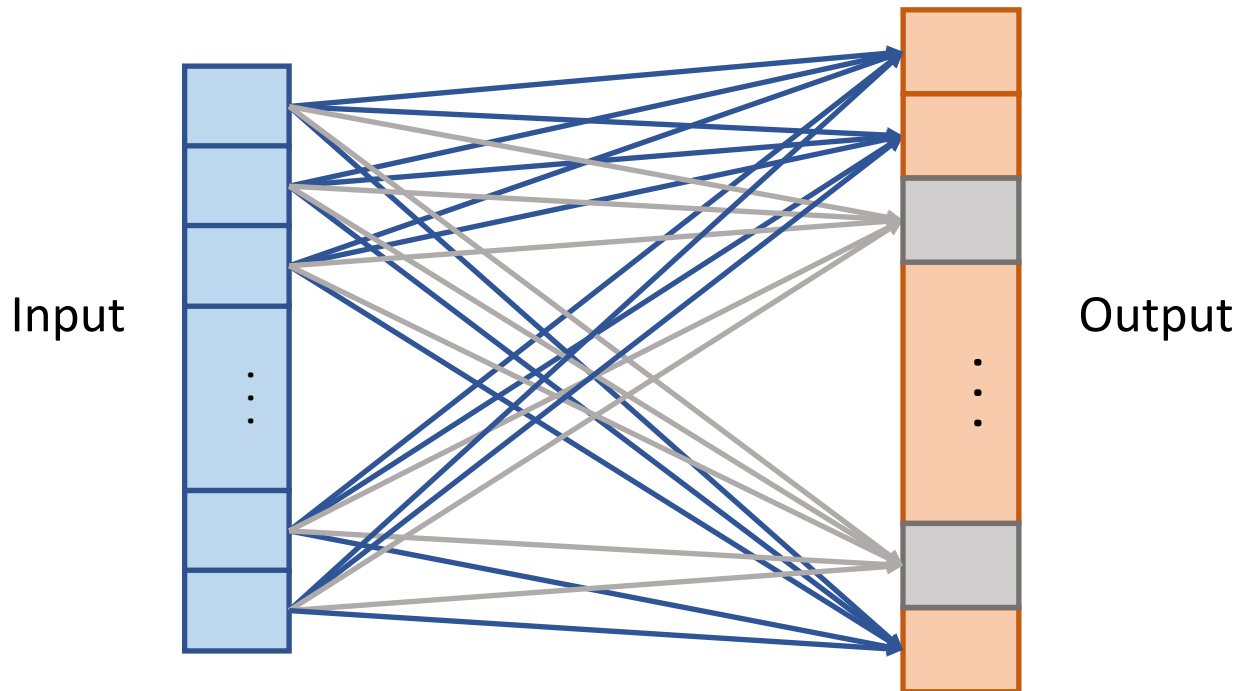
- It connects all its inputs to all the outputs
- It is usually used as the last layers of a network (before classification)



Convolutional Neural Networks

Drop-out layer

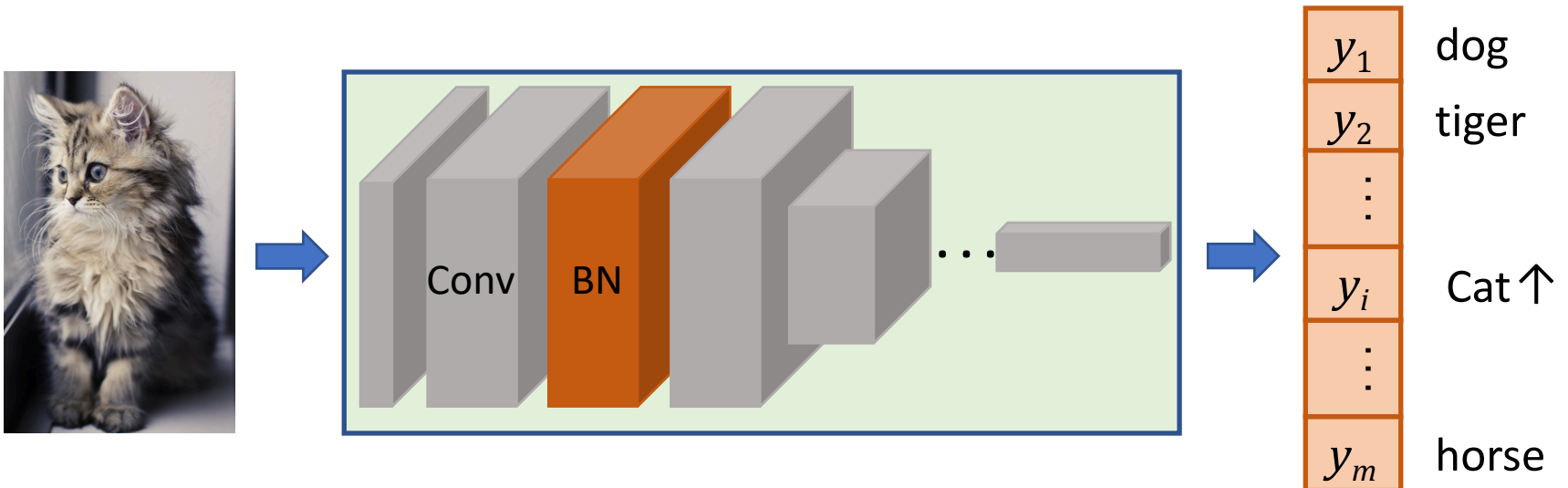
- It randomly drops out (sets to 0) neurons during training
- This prevents over-fitting (regularisation)



Convolutional Neural Networks

Batch-Normalisation layer

- It normalises its input across mini-batches
- It stabilises the network against the changes in the distribution of its input
- It helps the network generalise better to test data (regularisation)



Convolutional Neural Networks

Batch-Normalisation layer

It subtracts the mini-batch mean from its input and divides it by the mini-batch standard deviation

$$\hat{x} = \frac{x - m}{s}$$

This is then shifted and scaled by two learnable parameters

$$x^{BN} = \hat{x} \cdot \gamma + \beta$$

The BN output has a distribution with mean β and standard deviation γ

Popular CNN Architectures

Popular CNN Architectures

CNN – VGG16

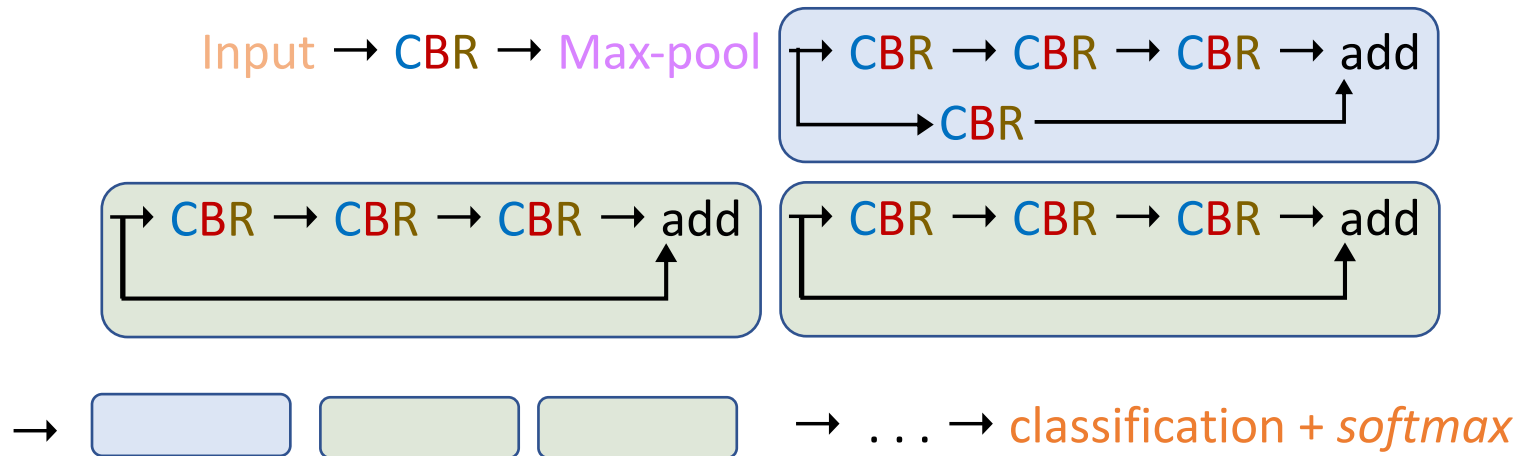
- It has 138 million parameters
- Its output layer has 1000 neurons

Input → Conv → ReLU → Conv → ReLU → Max-pool
→ Conv → ReLU → Conv → ReLU → Max-pool
→ Conv → ReLU → Conv → ReLU → Conv → ReLU → Max-pool
→ Conv → ReLU → Conv → ReLU → Conv → ReLU → Max-pool
→ FC → ReLU → FC → ReLU → classification + softmax

Popular CNN Architectures

CNN – ResNet

- It uses residual (shortcut) connections
- It has increased the number of layers (152)
- It has 26 million parameters (CBR: Conv → BN → ReLU)



Popular CNN Architectures

CNN for image classification

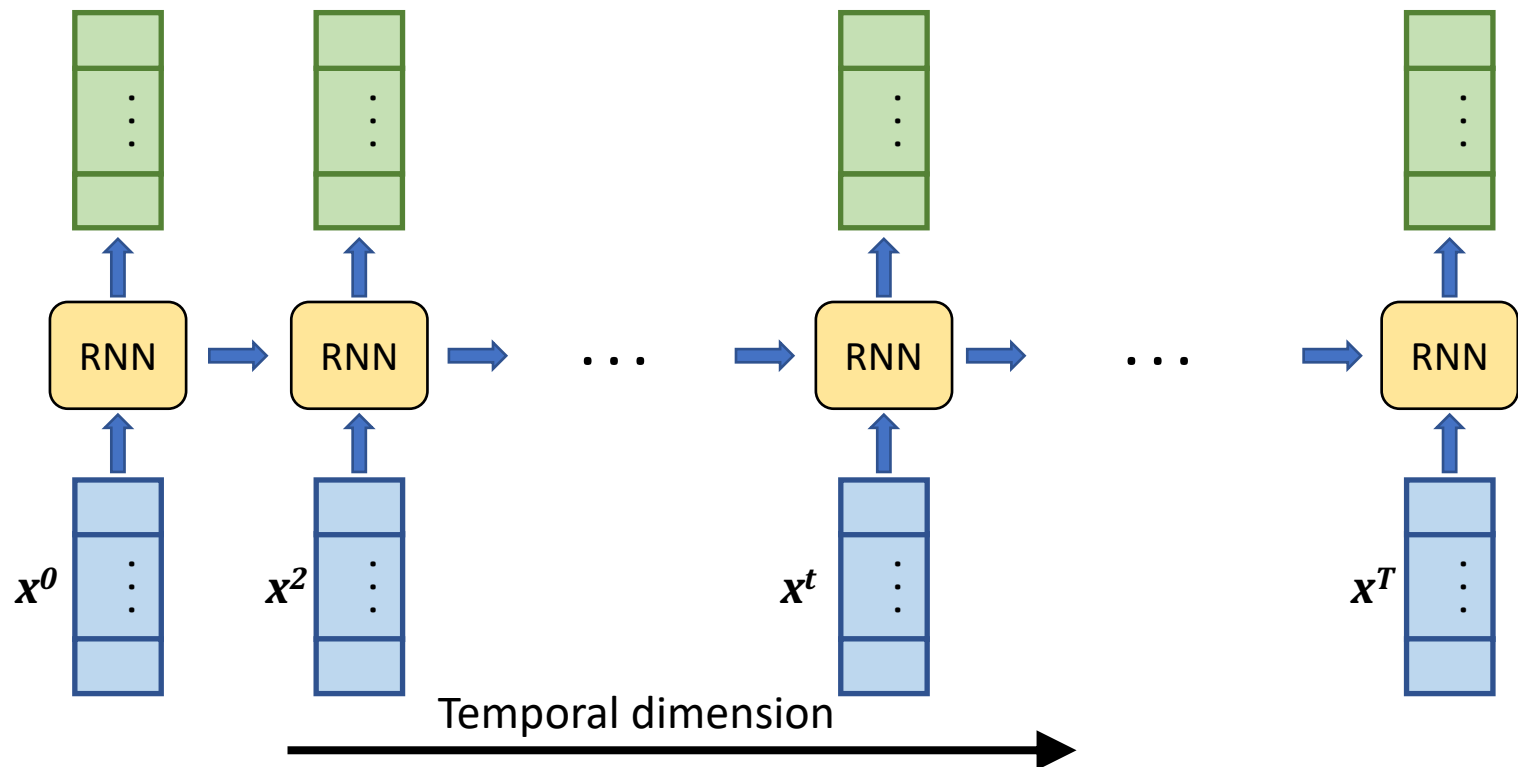
- AlexNet
- Inception-v1/ v2/ v4
- Inception-ResNets
- Xception
- DenseNet
- ResNeXt

Other Popular Neural Networks

Other Popular Neural Networks

Recurrent Neural Networks (RNN)

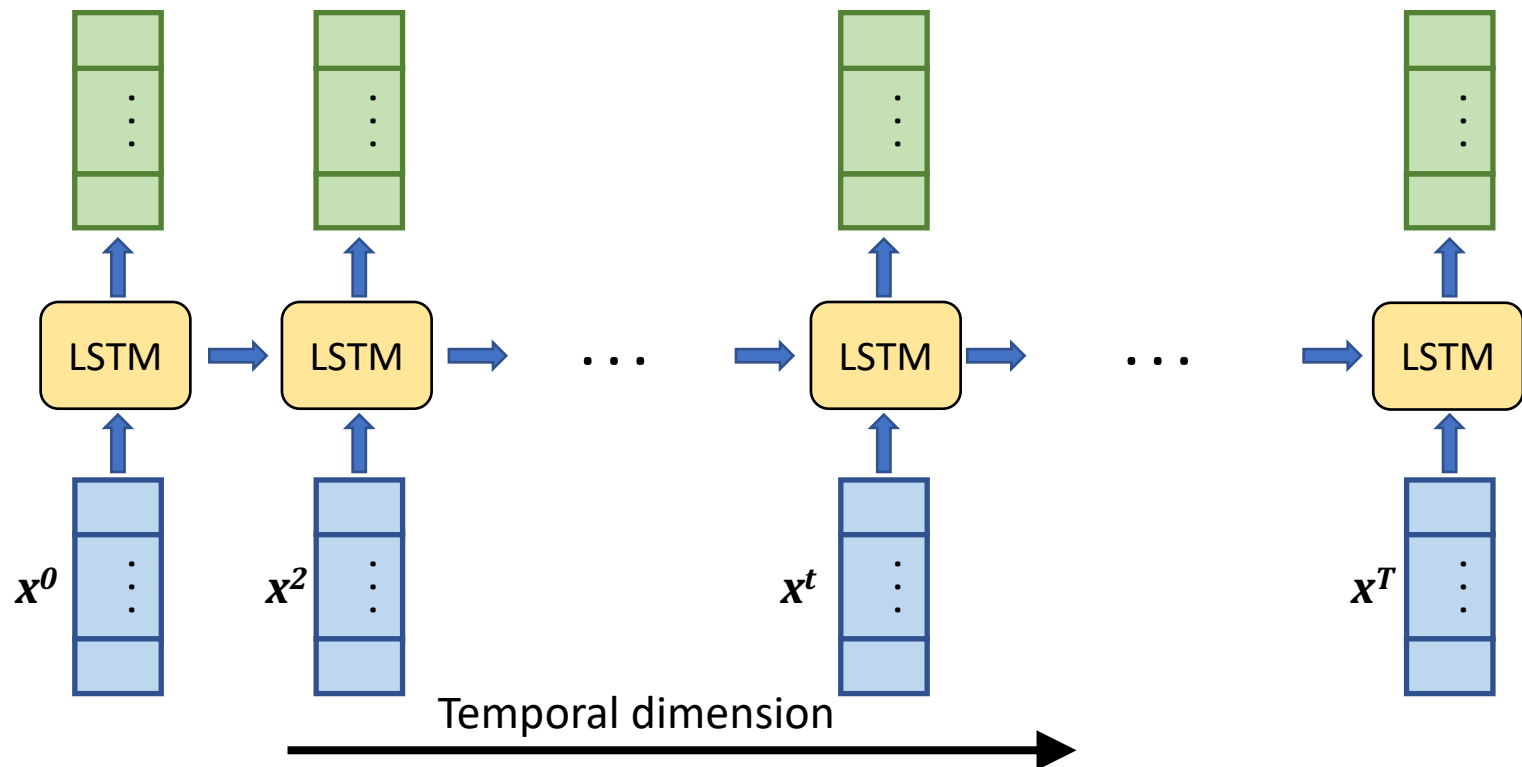
- They process sequences of data
- They back-propagate loss through time



Other Popular Neural Networks

Long Short-Term Memory Networks (LSTM)

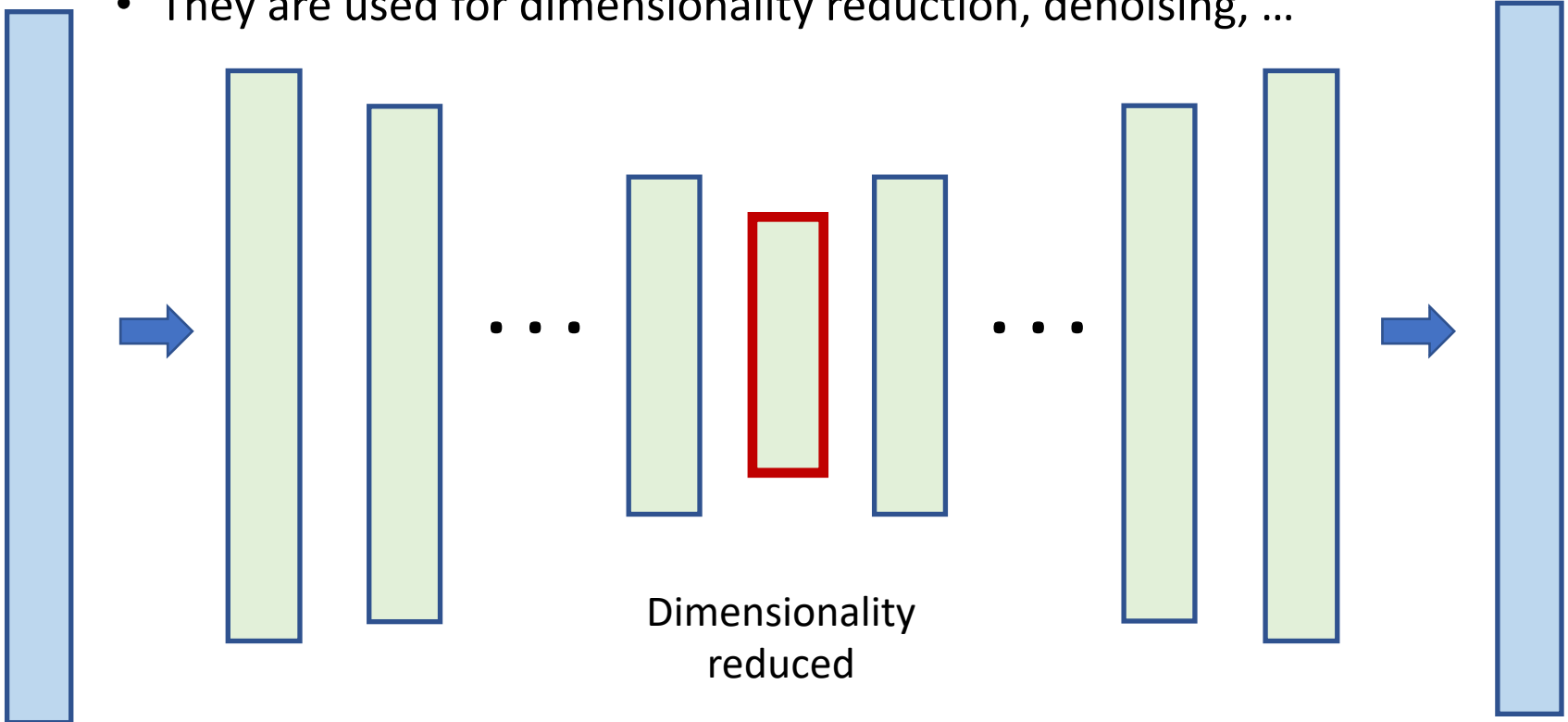
- They are a special type of Recurrent Networks
- They can store information in their memory for long durations



Other Popular Neural Networks

Auto-Encoders

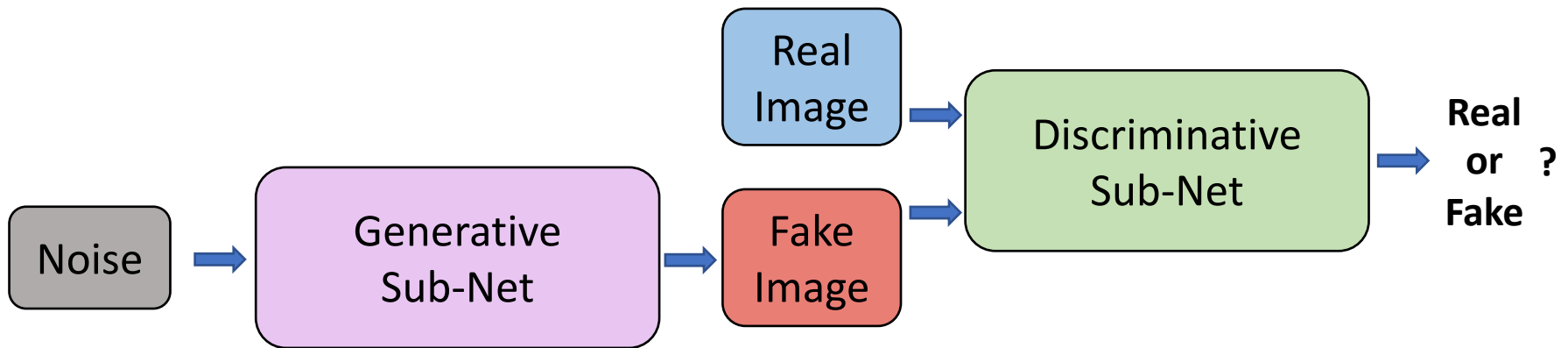
- They reconstruct their input
- They are used for dimensionality reduction, denoising, ...



Other Popular Neural Networks

Generative Adversarial Networks (GAN)

- They are generative models – e.g. when trained they can generate fake images out of noise
- They have two sub-network (discriminative and generative)
- The two sub-networks compete against each other (adversarial)



Summary and Conclusion

- We saw different types of layers in a CNN

Convolutional, Activation, Pooling

Fully-Connected, Drop-out, Batch Normalisation, ...

- We saw some popular neural networks
 - CNN (VGG and ResNet)
 - RNN and LSTM
 - Auto-Encoder
 - GAN

Thanks for your attention